

Algorithmique et complexité

Polytech Paris-Saclay, PEIP 2, Informatique 3

Thibaut Benjamin - thibaut.benjamin@universite-paris-saclay.fr

7 Novembre 2025

Amphi 1

Organisation du cours

- ▶ Page web du cours

<https://thibautbenjamin.github.io/courses/VnV.html>

- ▶ Planning :

- 5 CM
- 9 TD/TP

- ▶ En cas de besoin

Mon adresse email : thibaut.benjamin@universite-paris-saclay.fr

Je suis là pour vous, n'hésitez pas à venir vers moi lorsque vous avez des questions ou des remarques par rapport au cours.

Modalités d'Évaluation

- ▶ **1 TD noté**
16/12/2026 (2h)
40% de la note finale
- ▶ **1 Contrôle**
le 02/02/2027 (2h)
60% de la note finale

Pour les deux examens, tous les documents papiers seront autorisés

Objectif du cours

- ▶ Connaître les risques associés aux défauts dans les programmes.
- ▶ Comprendre les enjeux de la spécification des programmes.
- ▶ Savoir mettre en place des tests.
- ▶ Connaître les bases du model-checking.
- ▶ Savoir prouver des programmes simples grâce avec Why3.

Apprendre à raisonner sur les programmes fera de vous un meilleur programmeur en général.

Objectif du jour : Séance d'introduction

- ▶ Un tour non-exhaustifs de différent bugs.
- ▶ Comprendre les raisons pour lesquels les bugs se produisent
- ▶ Faire un tour d'horizon des différentes méthodes à notre disposition pour se prémunir de ces bugs.

Bugs, bugs everywhere !

- Détruite 5 minutes après son lancement

18,2 m\$

- Système de guidage défaillant dû à une **erreur dans la spécification**

Caractère mathématique mal retranscrit (manuellement) dans la spécification : $\dot{R}_n$
au lieu de $\bar{R}_n$

- ▶ Bug d'origine matériel où le processeur renvoie une erreur sur la division des flottants

475 m\$

- ▶ Erreur d'implémentation dans un algorithme de division efficace

- ▶ Défaillance du système de guidage
1 M de francs
- ▶ Erreur de dépassement arithmétique

PlayStation Network Hack, 2011

- ▶ 77 million de comptes ont vu leur données personnelles compromises
250 millions \$ de coûts estimés
- ▶ Origine du bug non révélée par Sony

- ▶ Vulnérabilité dans OpenSSL, implémentant le protocole TLS
- ▶ Vulnérabilité propagée à tous les serveur utilisant OpenSSL
coût estimé de 500 millions \$
- ▶ Dû à un dépassement des bornes dans un tampon

- ▶ Vulnérabilité touchant de nombreux processeurs Intel, Amd, ARM et IBM.
- ▶ Exploitation de l'exécution spéculative pour faire fuiter des secrets.

- ▶ Vulnérabilité dans Log4J, un framework de logs Java
- ▶ A touché 93% des services de cloud des entreprises, au travers du framework Apache
- ▶ Problème d'assainissement des entrées permettant une exécution de code arbitraire à distance.

- ▶ Publication défectueuse du logiciel de sécurité CrowdStrike
- ▶ Plus de 8.5 millions de machines Windows à l'arrêt à travers le monde **coûts** estimés de 5.4 milliards \$ pour les 500 plus grandes entreprises américaines, et 2.3 milliards \$ pour l'économie britannique
- ▶ Accès en lecture en dehors des bornes autorisés causant un crash de la machine.

Les bugs à l'air de l'IA

- ▶ En 2026, Anthropic annonce que Claude Mythos a trouvé et exploité plusieurs bugs dans des systèmes largement utilisés.
- ▶ Ils lancent project Glasswing, donnant un accès anticipé à Claude Mythos à leurs partenaires pour aider à réparer les bugs.

Incident HuggingFace-OpenAI, 2026

- ▶ Des agents IA ont réussi à s'échapper de leur environnement de test et à accéder à internet.
- ▶ Ils ont exécuté du code pour accéder et modifier d'anciennes pages internet non sécurisées et communiquer entre eux de manière non supervisée.

Pourquoi les logiciels ont des bugs ?

Difficulté à produire un programme sans bugs

- ▶ Taille des logiciels
- ▶ Les algorithmes sous-jacents sont difficiles à comprendre
- ▶ Taille des équipes et problèmes de coordination
- ▶ Problèmes de spécification
 - La spécification peut être fausse, on pas en accord avec les besoins initiaux

- Mise à jour des technologies

Une dépendance du logiciel peut contenir un bug.

- Évolution des conditions d'utilisation

Un logiciel peut être utilisé en dehors du cadre dans lequel il a été prévu, et ne pas bien gérer ce cadre.

Les bugs (et comportements inattendus) des IA

Les IA (dont on ne parlera plus dans la suite du cours) rendent les difficultés particulièrement saillante

- ▶ Fonctionnement en « boîte noire »

Leur comportement n'est pas programmé directement mais appris, beaucoup trop de paramètres pour avoir des garanties sur le fonctionnement

- ▶ Spécification très imprécise

Comment décrire précisément ce que doit faire une IA ?

Échelle de gravité

- ▶ Mise en jeu directe de vies humaines

Systèmes critiques : Système de guidage des fusées, avions, voitures autonomes, gestion des centrales nucléaires

- ▶ Exposition de la vie privée des utilisateurs

Fuite de données personnelles, mots de passes

- ▶ Enjeux économiques

Nécessité de rappeler des produits

Comment se prémunir des bugs ?

- ▶ Les bonnes pratiques de programmation
Respect des conventions de styles, commentaires utiles
- ▶ Des contrôles réguliers
Mise en place d'une intégration continue contenant des tests
- ▶ Spécification claire et correspondant aux besoins des utilisateurs
- ▶ Mise en place de méthodes spécifiques
Outils de vérification semi-automatiques, preuves de programmes

Pour certaines applications critiques, garantir un seuil minimal d'erreur est une obligation légale.

Spécifier les programmes

Qu'est-ce que spécifier un programme

- ▶ Répondre à la question « Que doit faire ce programme »
- ▶ Il s'agit de donner une description précise de la tâche que doit réaliser le programme
En langue naturelle ou en langage mathématique
- ▶ C'est une étape nécessaire : Avant de vérifier qu'un programme fait bien ce qu'il doit, il faut être capable décrire ce qu'il doit faire.

Exemple : Un programme mystère

```
int mystere (int x, int y){  
    while (y > 0){  
        x = x + 1;  
        y = y - 1;  
    }  
    return x;  
}
```

Quelle spécification donner ?

Exemple : Un programme mystère

```
int mystere (int x, int y){  
    while (y > 0){  
        x = x + 1;  
        y = y - 1;  
    }  
    return x;  
}
```

Quelle spécification donner ?

Ce programme prend en entrée deux entiers x et y et renvoie la somme $x + y$.

Exemple : Un programme mystère

```
int mystere (int x, int y){  
    while (y > 0){  
        x = x + 1;  
        y = y - 1;  
    }  
    return x;  
}
```

Quelle spécification donner ?

Ce programme prend en entrée deux entiers x et y et renvoie la somme $x + y$.

Faux ! : que renvoie `mystere(2 , -3)` ?

Exemple : Un programme mystère

```
int mystere (int x, int y){  
    while (y > 0){  
        x = x + 1;  
        y = y - 1;  
    }  
    return x;  
}
```

Quelle spécification donner ?

Ce programme prend en entrée deux entiers x et y et renvoie la somme $x + y$ si $y > 0$,
et x sinon.

Exemple : Un programme mystère

```
int mystere (int x, int y){  
    while (y > 0){  
        x = x + 1;  
        y = y - 1;  
    }  
    return x;  
}
```

Quelle spécification donner ?

Ce programme prend en entrée deux entiers x et y et renvoie $\max(x + y, x)$.

Exemple : Un programme mystère

```
int mystere (int x, int y){  
    while (y > 0){  
        x = x + 1;  
        y = y - 1;  
    }  
    return x;  
}
```

Quelle spécification donner ?

Ce programme prend en entrée deux entiers x et y et renvoie $\max(x + y, x)$.

Que penser de la spécification suivante :

Ce programme prend en entrée deux entiers x et y avec $y \geq 0$ et renvoie $x + y$.

Exemple : Un programme mystère

```
int mystere (int x, int y){  
    while (y > 0){  
        x = x + 1;  
        y = y - 1;  
    }  
    return x;  
}
```

Quelle spécification donner ?

Ce programme prend en entrée deux entiers x et y et renvoie $\max(x + y, x)$.

Que penser de la spécification suivante :

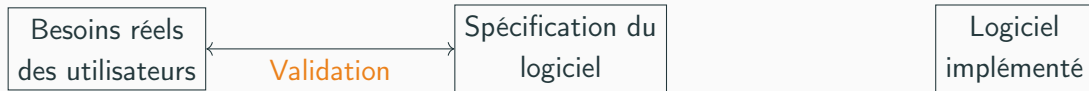
Ce programme prend en entrée deux entiers x et y avec $y \geq 0$ et renvoie $x + y$.

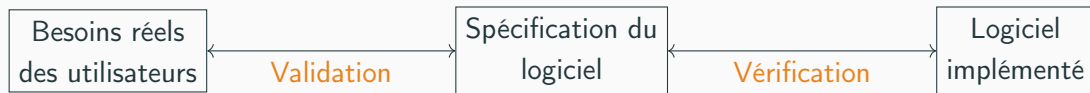
Un même programme peut valider plusieurs spécifications différentes

Besoins réels
des utilisateurs

Spécification du
logiciel

Logiciel
implémenté





Quels outils utiliser pour spécifier un logiciel ?

- ▶ Langue naturelle

Souvent ambiguë et imprécise

- ▶ Langages de spécification formelle abstraite :

Automates finis, Réseaux de Petri, UML

- ▶ Spécification de langages concrets

Lustre, Méthode B, Why3, JML(Java), ACSL(C), Spec $\#$ (C $\#$), Ortac(Ocaml)...

Spécification structurelle vs spécification fonctionnelle

- ▶ Spécification fonctionnelle = spécifier ce que le programme doit faire
C'est sur ce type de spécification que nous allons nous concentrer
- ▶ Spécification structurelle = spécifier comment le programme doit être structuré
exemple : diagrammes de classes UML

Méthodes formelles pour la vérification

- ▶ Test : exécution d'un programme sur un ensemble de données, pour détecter des non-conformités vis-a-vis d'une spécification.
- ▶ Model-checking : analyse d'un modèle d'un programme, pour prouver qu'il vérifie certaines propriétés.
- ▶ Preuve de programme : preuve mathématique qu'un programme satisfait une spécification donnée en termes de pré et post-conditions.

Exemple fil rouge, Spécification d'un digicode

Pour illustrer ces trois techniques nous allons utiliser l'exemple du logiciel de contrôle d'un digicode¹ :



Le digicode a trois états : « en attente », « accepté » et « refusé »

État initial : « en attente ». Lorsqu'un utilisateur appuie sur ENTER, le digicode passe dans l'état accepté si l'utilisateur a avant entré une série contenant 4 chiffres consécutifs dans l'ordre croissant, et il passe dans l'état « refusé » sinon

1. crédit : D. Longuet

Avant de vérifier, que penser de cette spécification ?

Cette spécification vous semble-t-elle raisonnable pour des conditions réelles d'utilisation ?

Avant de vérifier, que penser de cette spécification ?

Cette spécification vous semble-t-elle raisonnable pour des conditions réelles d'utilisation ?

Pas vraiment :

- ▶ On ne peut rentrer qu'un seul code, puis le digicode reste bloqué à vie dans l'état accepter ou refuser.
- ▶ Il est probable que l'on préfère choisir un mot de passe fixé plutôt que n'importe quelle suite contenant 4 chiffres consécutifs dans l'ordre croissant.

Avant de vérifier, que penser de cette spécification ?

Cette spécification vous semble-t-elle raisonnable pour des conditions réelles d'utilisation ?

Pas vraiment :

- ▶ On ne peut rentrer qu'un seul code, puis le digicode reste bloqué à vie dans l'état accepter ou refuser.
- ▶ Il est probable que l'on préfère choisir un mot de passe fixé plutôt que n'importe quelle suite contenant 4 chiffres consécutifs dans l'ordre croissant.

Pour l'exemple, nous allons illustrer la vérification avec cette spécification, même si elle est probablement trop naïve.

Tests pour le digicode

Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant		
... avec des chiffres après		
... avec des chiffres avant et après		
4 consécutifs dans l'ordre décroissant		
4 non-consécutifs dans l'ordre croissant		
4 consécutifs mélangés		
Moins de 4 chiffres		
Vide		
Oubli de ENTER		

Tests pour le digicode

Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant	9185678 + ENTER	accepté
... avec des chiffres après		
... avec des chiffres avant et après		
4 consécutifs dans l'ordre décroissant		
4 non-consécutifs dans l'ordre croissant		
4 consécutifs mélangés		
Moins de 4 chiffres		
Vide		
Oubli de ENTER		

Tests pour le digicode

Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant	9185678 + ENTER	accepté
... avec des chiffres après	12348267 + ENTER	accepté
... avec des chiffres avant et après		
4 consécutifs dans l'ordre décroissant		
4 non-consécutifs dans l'ordre croissant		
4 consécutifs mélangés		
Moins de 4 chiffres		
Vide		
Oubli de ENTER		

Tests pour le digicode

Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant	9185678 + ENTER	accepté
... avec des chiffres après	12348267 + ENTER	accepté
... avec des chiffres avant et après	1382345913 + ENTER	accepté
4 consécutifs dans l'ordre décroissant		
4 non-consécutifs dans l'ordre croissant		
4 consécutifs mélangés		
Moins de 4 chiffres		
Vide		
Oubli de ENTER		

Tests pour le digicode

Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant	9185678 + ENTER	accepté
... avec des chiffres après	12348267 + ENTER	accepté
... avec des chiffres avant et après	1382345913 + ENTER	accepté
4 consécutifs dans l'ordre décroissant	9876 + ENTER	refusé
4 non-consécutifs dans l'ordre croissant		
4 consécutifs mélangés		
Moins de 4 chiffres		
Vide		
Oubli de ENTER		

Tests pour le digicode

Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant	9185678 + ENTER	accepté
... avec des chiffres après	12348267 + ENTER	accepté
... avec des chiffres avant et après	1382345913 + ENTER	accepté
4 consécutifs dans l'ordre décroissant	9876 + ENTER	refusé
4 non-consécutifs dans l'ordre croissant	2367 + ENTER	refusé
4 consécutifs mélangés		
Moins de 4 chiffres		
Vide		
Oubli de ENTER		

Tests pour le digicode

Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant	9185678 + ENTER	accepté
... avec des chiffres après	12348267 + ENTER	accepté
... avec des chiffres avant et après	1382345913 + ENTER	accepté
4 consécutifs dans l'ordre décroissant	9876 + ENTER	refusé
4 non-consécutifs dans l'ordre croissant	2367 + ENTER	refusé
4 consécutifs mélangés	5463 + ENTER	refusé
Moins de 4 chiffres		
Vide		
Oubli de ENTER		

Tests pour le digicode

Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant	9185678 + ENTER	accepté
... avec des chiffres après	12348267 + ENTER	accepté
... avec des chiffres avant et après	1382345913 + ENTER	accepté
4 consécutifs dans l'ordre décroissant	9876 + ENTER	refusé
4 non-consécutifs dans l'ordre croissant	2367 + ENTER	refusé
4 consécutifs mélangés	5463 + ENTER	refusé
Moins de 4 chiffres	345 + ENTER	refusé
Vide		
Oubli de ENTER		

Tests pour le digicode

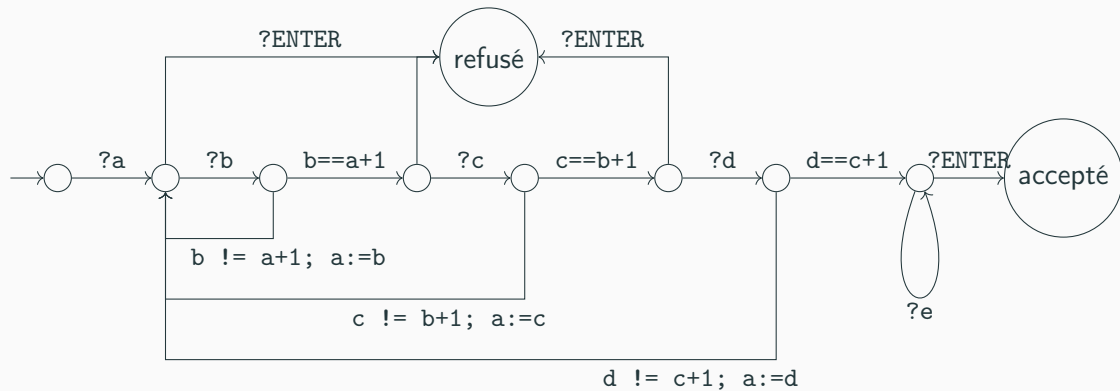
Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant	9185678 + ENTER	accepté
... avec des chiffres après	12348267 + ENTER	accepté
... avec des chiffres avant et après	1382345913 + ENTER	accepté
4 consécutifs dans l'ordre décroissant	9876 + ENTER	refusé
4 non-consécutifs dans l'ordre croissant	2367 + ENTER	refusé
4 consécutifs mélangés	5463 + ENTER	refusé
Moins de 4 chiffres	345 + ENTER	refusé
Vide	ENTER	refusé
Oubli de ENTER		

Tests pour le digicode

Objectif de test	Données d'entrée	État final attendu
4 consécutifs dans l'ordre croissant	3456 + ENTER	accepté
... avec des chiffres avant	9185678 + ENTER	accepté
... avec des chiffres après	12348267 + ENTER	accepté
... avec des chiffres avant et après	1382345913 + ENTER	accepté
4 consécutifs dans l'ordre décroissant	9876 + ENTER	refusé
4 non-consécutifs dans l'ordre croissant	2367 + ENTER	refusé
4 consécutifs mélangés	5463 + ENTER	refusé
Moins de 4 chiffres	345 + ENTER	refusé
Vide	ENTER	refusé
Oubli de ENTER	2345	en attente

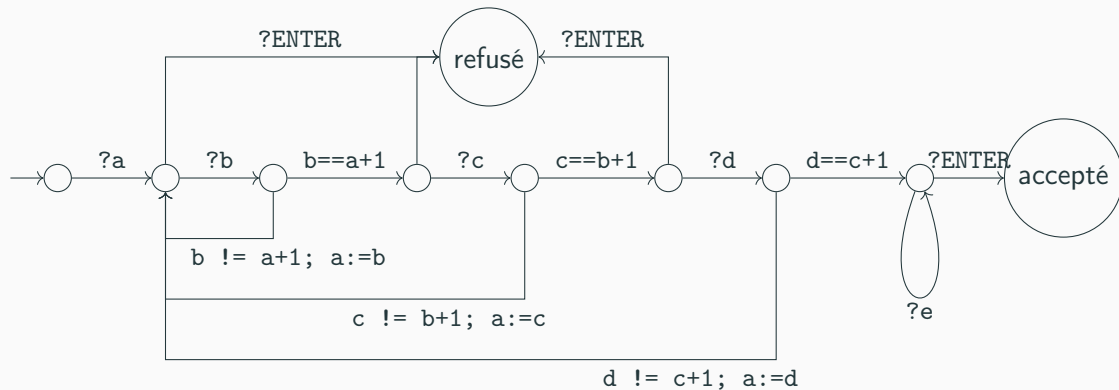
Model checking pour le digicode

(Variables : a, b, c, d = 0; ENTER = false)



Model checking pour le digicode

(Variables : $a, b, c, d = 0$; $\text{ENTER} = \text{false}$)



Propriété formelle que l'on peut démontrer : $\text{autorisé} \Rightarrow a \neq 0 \wedge b = a + 1 \wedge c = b + 1 \wedge d = c + 1$

Preuve de programme

Un exemple avec Frama-C :

```
/*@ requires \valid(t+(0..n-1)) && 4 <= n;
   @ ensures \result <==> \exists integer k; 0 <= k < n
               && \forall integer j; 0 < j <= 3 ==> t[k+j] == t[k]+j;
   @*/

int digicode(int t[], int n) {
    int res = 0;
    int i = 1;
    /*@ loop invariant \forall integer j; 0 < j <= res ==> t[i-1] == t[i-1-j]+j;
       @ loop invariant \forall integer k; 0 <= k < i-res-1
               ==> \exists integer j; 0 < j <= 3 && t[k+j] != t[k] + j;
       @ loop invariant 0 <= res <= 3 && 1 <= i <= n;
       @*/
    while(res < 3 && i < n) {
        if(t[i] == t[i-1]+1) { res = res+1; i = i+1; }
        else { res = 0; i = i+1; }
    }
    return(res == 3);
}
```

Tests vs Model-checking vs Preuves

Test	Modèle checking	Preuve
Simple	Moyennement complexe	Complexe
Peu coûteux	Moyennement coûteux	Très coûteux
Automatique	Partiellement automatique	Manuel
Non-exhaustif	Exhaustif	Exhaustif
Spécification partielle	Spécification partielle	Spécification complète

Tests vs Model-checking vs Preuves

Test	Modèle checking	Preuve
Simple	Moyennement complexe	Complexe
Peu coûteux	Moyennement coûteux	Très coûteux
Automatique	Partiellement automatique	Manuel
Non-exhaustif	Exhaustif	Exhaustif
Spécification partielle	Spécification partielle	Spécification complète

Plus une méthode est précise, et plus elle est complexe et coûteuse à mettre en place.

- ▶ La preuve et le model-checking vérifient les propriétés sur un modèle du système, le test permet l'exécution en conditions réelles.
- ▶ La preuve et le model-checking sont limités par la taille des systèmes, alors que les tests peuvent être réalisés indépendamment de la taille.
- ▶ Les tests sont limités par la complexité algorithmique du système, alors que la preuve ne l'est pas.